

На правах рукописи

Перминов Илья Валентинович

**Алгоритмические и программные средства
компрессии текстур на графических процессорах**

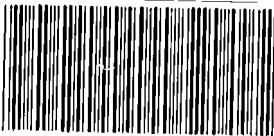
Специальность 05.13.11 —

«Математическое и программное обеспечение вычислительных
машин, комплексов и компьютерных сетей»

Автореферат

диссертации на соискание учёной степени
кандидата технических наук

27 НОЯ 2014



005555759

Санкт-Петербург — 2014

Работа выполнена в Санкт-Петербургском национальном исследовательском университете информационных технологий, механики и оптики.

Научный руководитель: **Тропченко Александр Ювенальевич**
доктор технических наук, профессор

Официальные оппоненты: **Терехов Андрей Николаевич**,
доктор физико-математических наук, профессор,
Санкт-Петербургский государственный университет,
заведующий кафедрой системного программирования

Белозерцев Александр Витальевич,
кандидат технических наук,
Санкт-Петербургский государственный университет
кино и телевидения, институт медиатехнологий,
и.о. заведующего кафедрой телевидения

Ведущая организация: Федеральное государственное бюджетное учреждение науки Институт прикладной математики им. М.В. Келдыша Российской академии наук.

Защита состоится 15 декабря 2014 г. в 18:30 часов на заседании диссертационного совета Д.212.227.06 при Университете ИТМО по адресу: 197101, г. Санкт-Петербург, Кронверкский пр., д. 49, конференц-зал центра интернет-образования.

С диссертацией можно ознакомиться в библиотеке Санкт-Петербургского национального исследовательского университета информационных технологий, механики и оптики по адресу: 197101, Санкт-Петербург, Кронверкский пр., д.49 и на сайте <http://fpro.ifmo.ru>.

Автореферат разослан «___» _____ 2014 года.

Ученый секретарь
диссертационного совета
Д.212.227.06



И.С. Лобанов

Общая характеристика работы

Актуальность темы. Принимая во внимание огромную вычислительную мощность современных графических процессоров в 5-8 Tflops, пропускная способность видеопамати свыше 300 ГБ/с зачастую становится узким местом. Кроме этого, подключение по шине PCIe со скоростью до 16 ГБ/с также может являться узким местом при доставке данных до устройства. Использование сжатия данных существенно снижает требования к объему видеопамати и пропускной способности интерфейса. Также серьезно снижаются энергозатраты на передачу данных.

В трёхмерной компьютерной графике основными классами данных, потребляющих около 80% ресурсов памяти и интерфейса, являются текстуры и G-буферы, которые также можно рассматривать как особый вид текстур. В простейшем случае текстуры представляют собой двухмерное изображение, накладываемое на трёхмерный объект, улучшающие визуальную детализацию трёхмерных объектов без усложнения геометрии. Кроме информации о цвете, текстура может содержать карту высот, нормалей, отражений и прочую информацию.

Широкое распространение получили различные технологии компрессии текстур. При этом текстура передается и хранится в памяти в сжатом виде, и распаковывается непосредственно внутри графического процессора (ГП), как правило, между кэш-памятью уровней L1 и L2. Такой подход позволяет не только уменьшить занимаемый текстурами объем памяти, но и снизить требования к пропускной способности. Кроме этого, сжатие позволяет снизить энергопотребление (за счёт снижения трафика на интерфейсе ГП↔память), что существенно для мобильных устройств.

Ввиду специфики применения и необходимости произвольного доступа к отдельным элементам текстуры, распространённые графические форматы, такие как JPEG и PNG, и универсальные алгоритмы сжатия не подходят для текстур. Поэтому в данной области применяются специально разработанные блочные схемы сжатия, такие как S3TC и PVRTC, обеспечивающие сжатие с потерями и фиксированный коэффициент сжатия.

Распаковка таких форматов не вызывает сложностей, однако сам процесс сжатия является нетривиальной задачей. Наилучшие результаты в отношении визуального качества дают методы, основанные на переборе. В связи с этим, важной практической характеристикой методов компрессии является их вычислительная сложность, в значительной мере определяющая скорость сжатия. Часть этапов компрессии может быть эффективно реализована на графических процессорах. Поэтому модификация существующих и создание новых

алгоритмов и программных средств для реализации на гетерогенных многоядерных системах может существенно ускорить процесс сжатия и уменьшить энергорасходы. Заметим, что в подобных системах универсальные и графические процессоры имеют доступ к общей памяти, и перенос вычислений с одного типа процессоров на другой имеет малые накладные расходы.

В современных системах визуализации все чаще используются динамически генерируемые текстурные данные, например, карты окружений для имитации отражения, которые могут обновляться на каждом кадре. Далее эти текстуры многократно используются для формирования текущего или последующих кадров, что делает актуальной задачу быстрого сжатия текстур в темпе генерации кадров, пусть и с некоторым снижением визуального качества. В идеальном случае такая текстура должна сжиматься собственными ресурсами графического процессора без обращения к видеопамати.

Однако к настоящему времени межкадровое (динамическое) сжатие текстур находится на начальной стадии развития. Отсутствуют быстрые алгоритмы сжатия, обеспечивающие достаточно высокое качество. Более того, архитектуры графических конвейеров и современные прикладные программные интерфейсы накладывают свои ограничения на доступ к текстурным данным, что затрудняет реализацию межкадрового сжатия.

В профессиональных сферах применения, таких как медицина, кинопроизводство, графический дизайн, обработка фотографий, зачастую используются мониторы с поддержкой глубины цвета 10 бит на цветовой канал и более и соответствующие форматы представления цвета. Полноценная поддержка подобных изображений при работе с трехмерной графикой также является важной задачей.

Предметом и объектом исследования в данной работе являются методы сжатия текстур, используемые в интерактивной компьютерной графике.

Целью данной работы является развитие методов компрессии текстур и создание программно-алгоритмических средств сжатия текстур, работающих как в гетерогенных системах, так и непосредственно на графическом процессоре.

Для достижения поставленной цели требуется решить следующие **задачи**:

1. Выполнить анализ существующих подходов к сжатию текстур, в том числе анализ применимости существующих подходов к сжатию текстур с высокой глубиной цвета, а также к динамическому сжатию на каждом кадре.
2. Разработать подходы, снижающие затраты на межкадровое сжатие текстур.
3. Разработать параллельную модификацию алгоритма сжатия текстур для исполнения на графических процессорах в темпе генерации кадров.

4. Разработать последовательно-параллельную модификацию алгоритма сжатия текстур для исполнения в гетерогенных системах, обеспечивающую значительное ускорение при сохранении высокого качества сжатия.
5. Провести экспериментальные исследования и оценить эффективность предложенных подходов.

Основные положения, выносимые на защиту:

1. Параллельный алгоритм и программная реализация сжатия текстур в формате DXT/BC1, имеющая высокую эффективность исполнения на графическом процессоре за счёт распараллеливания процесса кодирования отдельного блока изображения и исключения операций доступа к памяти, приводящим к конфликтам.
2. Метод сжатия текстур, позволяющий использовать существующие форматы для хранения текстур с высокой глубиной цвета (более 8 бит на компоненту) за счёт изменения этапа квантования и деквантования ключевых цветов.
3. Методы снижения затрат на межкадровую компрессию текстур на платформах графических процессоров за счёт исключения избыточных операций копирования данных и добавления дополнительной программируемой стадии в графический конвейер, позволяющие уменьшить объём записываемых в память данных до 8 раз.
4. Последовательно-параллельный алгоритм и программная реализация сжатия текстур в формате ASTC, нацеленная на одновременную работу алгоритма на ядрах ЦП и ГП в гетерогенных системах с общей виртуальной и физической памятью и обеспечивающая тем самым повышение производительности в 3–5 раз.

Научная новизна:

1. Разработана параллельная модификация алгоритма сортировки, нацеленная на работу на графическом процессоре и позволяющая исключить повторяющиеся элементы. Предлагаемый подход характеризуется отсутствием дивергенции потоков исполнения (за счёт отсутствия условных переходов) и позволяет резко снизить количество конфликтов при обращении к разделяемой памяти.
2. Предложен метод работы с небольшими массивами на графическом процессоре, позволяющий размещать массивы в регистровой памяти, даже при обращении различных потоков к элементам по разным индексам.

3. Предложен критерий точности сохранения градиентных цветов в сжатом блоке, основанный на том, что сжатие предполагается допустимым для заданной глубины цвета n бит, если равномерный вертикальный или горизонтальный градиент с шагом c_n , минимальным для данной глубины цвета, может быть сжат с максимальной ошибкой e не превышающей c_n .
4. На основе данного критерия предложена модификация соответствующей схемы сжатия для сохранения текстур с высокой глубиной цвета.
5. Предложена модификация структуры конвейера тайловых графических архитектур, позволяющая производить обработку любых текстур до их записи в память.
6. Предложен метод снижения затрат на межкадровую компрессию текстур за счёт избавления от избыточных операций на уровне программного интерфейса (API).
7. Разработана последовательно-параллельная модификация алгоритма и программная реализация сжатия текстур в формате ASTC, использующая особенности новейших гетерогенных архитектур для ускорения процесса компрессии.

Практическая значимость диссертационной работы состоит в увеличении производительности текстурных компрессоров, снижении требований к подсистеме видеопамати, повышении производительности и энергоэффективности вычислительной системы при работе с динамически изменяемыми текстурами.

Методы исследования включают в себя методы цифровой обработки изображений, компьютерной графики, дискретной математики, теории алгоритмов.

Достоверность научных положений, полученных в диссертации, подтверждается программной реализацией предложенных методов, экспериментальными исследованиями, а также внедрением результатов работы на практике.

Апробация работы. Основные результаты работы докладывались на следующих конференциях: III научно-практическая конференция молодых ученых "Вычислительные системы и сети (Майоровские чтения)" (Санкт-Петербург, 2011), I и II Всероссийский конгресс молодых ученых (Санкт-Петербург, 2012–2013), VI Международная конференция "Параллельные вычисления и задачи управления" РАСО'2012 (Москва, ИПУ РАН, 2012), IV Московский суперкомпьютерный форум МСКФ-2013 (Москва, 2013), 23я Международная Конференция по Компьютерной Графике и Зрению GraphiCon'2013 (Владивосток, 2013).

Внедрение результатов работы. Материалы диссертации используются в качестве рабочих документов в корпорации AMD для проектирования и вери-

фикации текстурных каналов, а также в компании Luxoft для отладки и оптимизации стека системного ПО для гетерогенных процессоров класса HSA, что подтверждается справками о внедрении.

Материалы диссертации использованы в НИР №713564 «Создание бесшовных технологий проектирования встраиваемых систем и систем на кристалле на основе реконфигурируемых архитектур» и НИР 610481 «Разработка методов и средств системотехнического проектирования информационных и управляющих вычислительных систем с распределенной архитектурой», а также в учебном процессе на кафедре ВТ (Университет ИТМО) в дисциплине «Теоретическая информатика» для магистров по направлению 230100.68.

Публикации. Основные результаты по теме диссертации изложены в 10 печатных изданиях, 3 из которых изданы в журналах, рекомендованных ВАК, 3 — в тезисах докладов, 1 статья опубликована на русском и английском языках в журнале, индексируемом в базе Scopus.

Личный вклад автора. Решение задач диссертации, разработанные алгоритмы и их реализация принадлежат лично автору.

Объем и структура работы. Диссертация состоит из введения, пяти глав, заключения и трёх приложений. Полный объем диссертации 138 страниц текста с 77 рисунками и 33 таблицами. Список литературы содержит 99 наименований.

Содержание работы

Во **введении** обосновывается актуальность исследований, проводимых в рамках данной диссертационной работы, формулируется цель, ставятся задачи работы, формулируется научная новизна и практическая значимость представляемой работы.

Первая глава посвящена подробному описанию существующих технологий сжатия текстур. Рассмотрены основные требования к методам сжатия и аппаратной реализации. Также рассмотрены существующие методы сжатия текстур в форматах BC1 и ASTC. Кроме этого приводится анализ архитектуры графических ускорителей и обзор технологии gprgri-вычислений OpenCL.

Отдельные точки раstra, составляющие текстуру, принято называть текселями (texel — texture element). Общая производительность системы визуализации очень сильно зависит от эффективности произвольного доступа к текселям. Эта особенность обуславливает основные свойства различных форматов сжатия текстур. Для текстурного сжатия характерно разбиение всего изображения на блоки фиксированного размера и независимое сжатие каждого блока. Примени-

тельно к компьютерной графике, для текстурной компрессии важны следующие свойства:

- *Произвольный доступ.* Необходимо, чтобы формат сжатой текстуры позволял осуществлять быстрый доступ к произвольному участку текстуры. В связи с этим, почти все известные компрессоры имеют *фиксированную степень сжатия*. Это, в свою очередь, означает *сжатие с потерями*.
- *Высокая скорость распаковки.* Время доступа к текстурам является крайне важным фактором, влияющим на общую производительность видеоподсистемы. Отсюда вытекают два следующих свойства:
 - Простота аппаратной реализации декодера
 - Выборка необходимых данных только за одно обращение к памяти
- *Высокая степень сжатия.* Степень сжатия принято выражать в среднем количестве бит на пиксель/тексель или bprp. Типичные значения bprp для текстурных компрессоров от 2bprp до 8bprp.
- *Блочный принцип сжатия.* Размер блока обычно составляет 4x4 текселя.
- *Минимальные искажения* сжатого изображения.

На данный момент, стандартом сжатия текстур на персональных компьютерах является семейство форматов S3TC, в которое входят форматы BC1-BC5 и новые форматы BC6H и BC7. Формат BC1 предполагает хранение в сжатом блоке пары базовых цветов C_0 и C_1 в формате RGB565 и таблицы двухбитных индексов, которые определяют в какой пропорции смешивать эти цвета при распаковке (Рис. 1). Декодер формирует локальную палитру, состоящую из четырех цветов, а индекс указывает какой из этих цветов необходимо выбрать. Таким образом, все цвета внутри блока квантуются на 4 уровня.

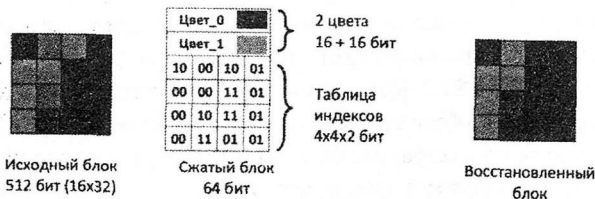


Рис. 1: Пример блока BC1.

Два дополнительных цвета C_2 и C_3 получаются путём интерполяции базовых цветов. При этом выделяется два типа блоков. Различаются типы блоков

между собой за счёт избыточности кодирования. Действительно, если в сжатом блоке поменять местами базовые цвета и пересчитать таблицу индексов, то при декомпрессии получится тот же самый блок. Блоки, у которых C_0 , интерпретированное как 16-ти битное целое число, больше чем C_1 , считаются блоками первого типа, все остальные блоки — блоками второго типа.

В большинстве реализаций дополнительные цвета C_2 и C_3 для блоков первого типа вычисляются по формулам:

$$C_2 = \frac{2}{3}C_0 + \frac{1}{3}C_1$$

$$C_3 = \frac{1}{3}C_0 + \frac{2}{3}C_1$$

Для блоков второго типа C_2 считается абсолютно прозрачным, а C_3 вычисляется по формуле:

$$C_3 = \frac{C_0 + C_1}{2}$$

Задача сжатия исходного блока в формат BC1 сводится к нахождению двух базовых цветов, позволяющих получить после распаковки блок, максимально близкий к оригиналу. Выбор базовых цветов является сложной задачей и оказывает сильное влияние на качество восстановленной текстуры.

Наиболее быстрые алгоритмы используют минимальные и максимальные значения цветов (FastDXT) в качестве базовых, или используют метод наименьших квадратов для их нахождения, что может приводить к значительным искажениям. Кодеки, рассчитанные на высокое качество (Squish, NVidia NVtt), не подходят для межкадрового сжатия ввиду относительно низкого быстродействия. За счёт использования итеративного подхода алгоритм LSDxt позволяет варьировать соотношение скорости и качества сжатия, позволяя использовать один алгоритм как для быстрого, так и для качественного сжатия. Процесс компрессии состоит из трёх основных этапов (Рис.2):

1. Линейная регрессия используется для получения начальных базовых цветов. При этом каждый цветовой канал обрабатывается независимо. Значения в красном, зеленом и синем каналах сортируются в порядке возрастания, дубликаты удаляются. Для каждого цветового канала применяется линейная регрессия: в качестве координаты Y используется интенсивность цвета, а в качестве X — индекс в отсортированном массиве. Однако точки, которые имеют очень близкие по значению цвета, будут “перекашивать” направление прямой, сильно снижая точность для “редких” цветов. Этот эффект корректируется путем предварительного приведения значения цветов от 32-битного к 16-битному представлению. Таким образом, близкие по значению цвета будут отброшены на этапе сортировки.

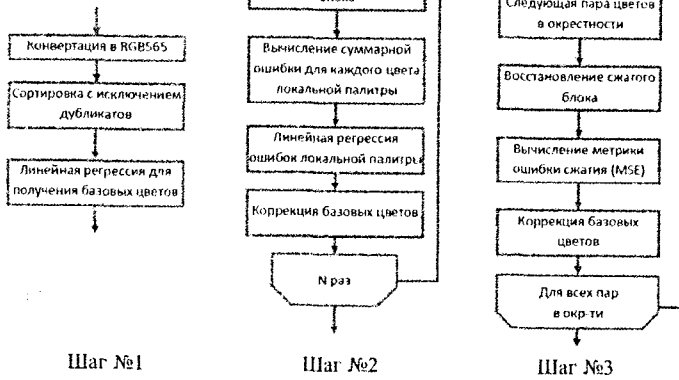


Рис. 2: Блок-схема алгоритма LSDxt.

- На втором этапе линейная регрессия используется для уточнения результата. По ключевым цветам воспроизводится сжатый блок, и линейная регрессия применяется к разности между восстановленным и оригинальным блоком. Второй проход линейной регрессии рассчитан на тот факт, что после сжатия внутри блока доступно только 4 различных цвета. Первый этап дает оптимальный результат для варианта, когда в сжатом блоке можно использовать любой цвет, находящийся на прямой, задаваемой двумя базовыми цветами. Используя эти цвета в качестве отправной точки, можно скорректировать направление прямой так, чтобы уменьшить разницу между доступными и исходными цветами. Данный процесс является итеративным.
- На последнем этапе производится перебор всех комбинаций цветов, находящихся в небольшой окрестности полученных результатов. Цвета, дающие минимальную ошибку, выбираются в качестве ключевых.

В формате BC7 размер сжатого блока увеличен до 128 бит, уровень сжатия равен 8bpp. Количество типов блоков расширено до 8, и тип блока кодируется явным образом первыми битами. Добавлена возможность сохранения до трёх пар базовых цветов. В зависимости от типа блока, размер индекса варьируется от 2 до 4 бит, а точность представления базовых цветов — от 4 до 7 бит. Любой блок исходного изображения можно закодировать именно тем типом блока, который позволяет получить наименьшую ошибку сжатия. Таким образом, формат BC7 существенно повышает визуальное качество сжатого изображения.

Новый формат сжатия ASTC, ещё не имеющий широкой поддержки со стороны аппаратного обеспечения, позволяет выделять внутри блока нецелое количество бит под отдельные значения индексов и интенсивностей базовых цветов за счёт использования алгоритма кодирования BISE. Это и другие улучшения, такие как поддержка трёхмерных текстур и текстур с широким динамическим диапазоном, возможность выбора уровня сжатия от 8.00 bpp до 0.89 bpp и до 0.59 bpp для трёхмерных текстур, делают формат ASTC самым гибким и перспективным форматом.

На основании проведённого анализа форматов сжатия текстур и архитектуры графических ускорителей в работе делаются следующие выводы:

- Все имеющиеся форматы нацелены на сжатие текстур с глубиной цвета 8 бит. Исключение составляют текстуры с широким динамическим диапазоном.
- Существующие техники межкадрового сжатия текстур неизбежно связаны с записью промежуточных данных в память, что снижает быстродействие независимо от используемого алгоритма и формата сжатия.
- Процесс компрессии в современные форматы, такие как BC7 и ASTC, является очень трудоёмким из-за высокой гибкости форматов и большого разнообразия типов блоков. Известна только одна реализация компрессора ASTC, но используемый алгоритм рассчитан на исполнение на ЦП.
- Архитектура ГП обладает существенными отличиями от других параллельных архитектур, влияющими на эффективность работы различных алгоритмов, и требует значительной переработки существующих алгоритмов или разработки новых версий.
- Особенности гетерогенной архитектуры HSA дают возможность работать с параллелизуемыми вычислительными задачами гораздо меньшего размера, по сравнению с традиционными архитектурами, за счёт снижения накладных расходов на вызов вычислительных фрагментов и отсутствия необходимости в копировании данных.

Во второй главе предлагается параллельная модификация алгоритма сжатия текстур LSDxt для работы на графическом процессоре.

Каждая инструкция в отдельном ядре ГП выполняется одновременно над целой группой элементов данных и может рассматриваться как исполнение вектора потоков, что соответствует концепции SIMD (Single Instruction Multiple Data). Размер вектора потоков для современных архитектур составляет 32 или 64 элемента, а количество ядер внутри одного графического процессора, способных

независимо исполнять различные векторные инструкции, исчисляется десятками. Для эффективного использования вычислительной мощности ГП, программа должна создавать тысячи параллельных потоков, а отдельные группы потоков отображаемые на аппаратные вектора, должны выполнять когерентные вычисления и когерентный доступ к памяти.

Большое количество потоков достигается естественным образом за счёт возможности независимого сжатия всех блоков исходного изображения. Однако реализация отдельных этапов сжатия привычным образом имеет низкую эффективность работы на ГП. В частности, на стадии сортировки на первом этапе алгоритма LSDxt требуется отсортировать массив A из 16 элементов, убрав повторяющиеся, и узнать количество уникальных элементов. При этом диапазон значений элементов известен заранее.

В работе предлагается алгоритм на основе сортировки подсчётом (parallel rank sort), при этом сложность алгоритма — $O(n)$, при n , не превышающем размера вектора потоков (Рис.3):

1. Временный массив B заполняется значениями, заведомо большими любого из исходных элементов A (обозначим их как " ∞ ").
2. Каждый поток подсчитывает ранг "своего" элемента. Под рангом понимается количество элементов строго меньше заданного.
3. Ранг используется как индекс для записи элемента в массив B . При этом в B останутся только уникальные значения и, возможно, несколько " ∞ ".
4. Шаги 2 и 3 повторяются для массива B . Таким образом, все уникальные значения сгруппируются в верхней части массива C .

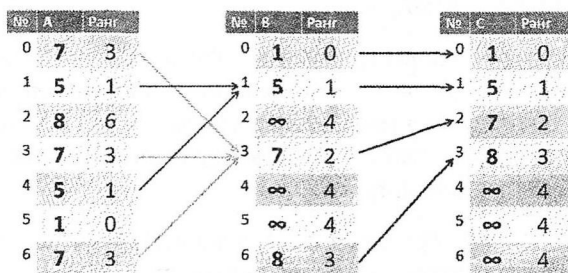


Рис. 3: Пример сортировки уникальных значений.

Производительность локальной разделяемой памяти сильно зависит от наличия конфликтов банков памяти. Они возникают, когда потоки в рамках одного вектора потоков обращаются к различным ячейкам одного банка памяти.

Банки локальной памяти чередуются с шагом 4 байта. Для минимизации количества конфликтов, в реализации алгоритма сортировки используются типы данных с размером 4 байта и цветовые компоненты текселей чередуются с пустыми элементами. Кроме этого, в алгоритме отсутствуют условные переходы, поэтому дивергенция потоков исполнения также отсутствует. Эксперименты показали, что предложенный параллельный алгоритм сортировки позволил существенно увеличить быстродействие этапа 1 (Таблица 1).

Описание	Время, мс
Сортировка методом вставки	13,89
Сортировка с подсчётом ранга	4,211
Сортировка с подсчётом ранга и исключением конфликтов банков памяти	2,699

Таблица 1: Время выполнения этапа №1 для текстуры размером 1920x1200

Алгоритм вычислений на этапах 2 и 3 также был модифицирован с целью выполнения всех расчётов непосредственно в регистровой памяти, так как она обладает наименьшей задержкой и наибольшей пропускной способностью на уровне десятков Tb/s. И хотя объёма регистровой памяти в десятки и сотни Kb на одно ядро ГП вполне достаточно для некоторых алгоритмов, имеются существенные архитектурные ограничения. Потоки одного вектора всегда выполняют одинаковые инструкции над одинаковыми векторными регистрами. К примеру, если индекс массива в операции $A[i] = 1$ не известен на момент компиляции, такой массив будет размещён в адресуемой памяти или операция будет развёрнута в цикл по каждому потоку с маскированием выполнения, что радикально снижает быстродействие подобной операции.

Для решения этой проблемы предлагается способ работы с небольшими массивами, заключающийся в использовании только индексов, известных на момент компиляции, путём перебора отдельных элементов массива. Для этого, вместо отдельного обращения, предлагается использовать цикл по всем элементам массива. Таким образом, фрагмент кода на языке OpenCL C:

```
1 Used [bestInd] = 1;
2 Bucket [bestInd] += MinError;
```

заменяется на

```
1 #pragma unroll
2 for(int i=0; i<MAX_I; i++) {
3     char pred = (bestInd == i) ? 1 : 0;
4     used[i] |= pred;
5     Bucket[i] += pred * MinError;
6 }
```

Использование директивы `#pragma unroll` позволяет компилятору развернуть цикл в последовательность операций без переходов. Подобный приём позволяет разместить все данные в регистрах и эффективно с ними работать. Сопоставление времени сжатия тестовой текстуры размером 512x512 с максимальным уровнем качества представлено в таблице 2. Из таблицы видно, что за счёт предлагаемой методики без изменения аппаратной конфигурации производительность увеличивается в 1.5–2 раза.

Модель ЦП/ГП	Время сжатия, с	Ускорение
Intel Core i3	1523	-
NVidia GTX 560	50.81	30x
NVidia GTX 560 (мод.)	30.7	50x
AMD HD 7970	16.48	92x
AMD HD 7970 (мод.)	6.8	223x

Таблица 2: Время сжатия тестовой текстуры

Третья глава посвящена исследованию возможности межкадрового (динамического) сжатия текстур на графическом процессоре безотносительно к конкретному формату и алгоритму сжатия.

Динамическое сжатие предполагает компрессию данных, появившихся во время или в результате процесса визуализации. В идеальном случае такая текстура должна сжиматься силами графического процессора без дополнительного обращения к памяти, однако это не всегда возможно. В работе выделяются три ситуации, различающиеся по возможностям осуществления межкадрового сжатия:

1. Текстура является результатом визуализации. Примерами являются карты окружений, а также G-буферы, используемые в методах отложенного освещения.
2. Текстура генерируется процедурным алгоритмом.
3. Данные уже имеются в памяти. К этой категории можно отнести создание составных текстур (compositing), а также транскодирование текстур.

Анализ работы ГП показал, что сжатие текстур первой категории невозможно на традиционных архитектурах ГП без предварительной записи в память, что переводит данный случай в категорию 3. Сжатие же текстур категорий 2 и 3 возможно силами пиксельного или вычислительного шейдера. Тем не менее современные программные интерфейсы (Direct3D 11.2 и OpenGL 4.4) не позволяют производить рендеринг в текстуры сжатого формата. Поэтому для сжатия

необходимо создавать отдельную текстуру, текстели которой будут хранить сжатые блоки. Результат работы шейдерной программы сохраняется в эту текстуру. После чего “сырые” данные копируются в созданную текстуру сжатого формата, которую уже можно использовать (Рис.4). Такой подход требует обязательную операцию копирования.

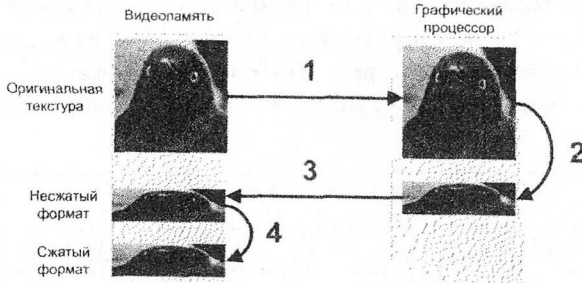


Рис. 4: Этапы межкадрового сжатия (1 — чтение из видеопамяти, 2 — сжатие, 3 — запись в текстуру несжатого формата, 4 — копирование в текстуру сжатого формата).

Для оценки затрат на копирование текстурного буфера были произведены эксперименты. В тестовой сцене с размером кадра 1920x1080 на каждом кадре сжималась текстура размером 1024x1024 в формат BC1 при помощи метода наименьших квадратов, и на экран выводились объекты, представляющие из себя кубы с использованием сжатой текстуры. Профилирование показало, что формирование одного кадра занимает 277 мкс, из которых 49 мкс приходится на сжатие текстуры, 10 мкс — на копирование текстурного буфера, а остальное время занимает очистка кадровых буферов и отрисовка тестовых объектов. При этом время отрисовки одного объекта тестовой сцены при использовании сжатых текстур снижается с 52 мкс до 45 мкс. Это объясняется снижением объёма считываемых из памяти текстурных данных при использовании компрессии. Таким образом, копирование занимает значительную часть времени формирования кадра и сопоставимо по времени с самим процессом сжатия.

В качестве решения в работе предлагается использовать псевдонимы буферов памяти (memory aliasing), т.е. объекты отображаемые в одинаковые адреса памяти. С точки зрения программных интерфейсов это может быть реализовано в виде расширения имеющегося механизма текстурных представлений (texture view). Таким образом, разные представления, имеющие сжатый и несжатый тип, будут указывать на одну и ту же область памяти. Это позволит полностью ликвидировать операцию копирования текстурного буфера.

Также был произведён анализ тайловых архитектур. В отличие от традиционных архитектур, вычисления производятся последовательно для отдельных тайлов (блоков) кадра. За счёт использования внутрикристального буфера цвета, содержимое тайла полностью формируется до записи в память.

Для подобных архитектур ГП в работе предлагается вариант сжатия любых текстур без предварительной записи в память. Для этого предлагается добавить в конвейер обработки дополнительную стадию “output shader”, запускающую шейдерную обработку перед финальной записью тайла в кадровый буфер (Рис. 5). Таким образом, объём передаваемых и записываемых в кадровый буфер данных уменьшается в несколько раз в зависимости от используемого формата сжатия.

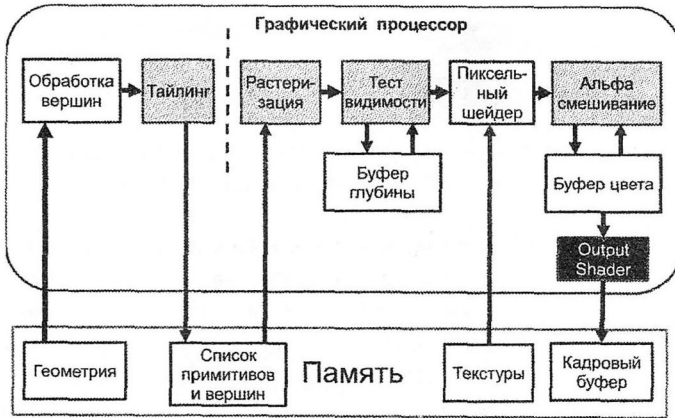


Рис. 5: Модифицированный тайловый конвейер с дополнительной стадией “output shader”.

Следует отметить, что в некоторых ГП используется компрессия буфера цвета. Примером служит технология ARM AFBC, обеспечивающая сжатие без потерь. По сравнению с таким подходом, предлагаемый метод имеет следующие преимущества:

- Более высокий уровень сжатия. Степень сжатия AFBC не превышает 1:2, в то время как текстурные компрессоры обеспечивают фиксированный уровень от 1:4 до 1:36.
- Новая стадия расширяет возможности архитектуры. Кроме сжатия могут быть реализованы эффекты, относящиеся к пост-обработке без дополнительных операций записи и чтения кадрового буфера. Примером может служить цветовая коррекция или наложение шума.

Четвертая глава посвящена компрессии текстур с высокой глубиной цвета.

Стандартным методом представления изображений в компьютерных системах является TrueColor, подразумевающий использование цветовой модели RGB с глубиной цвета 8 бит, поэтому интенсивности каждого цветового канала (красного, зеленого и синего) могут принимать значения от 0 до 255. Хотя приложения могут использовать для хранения и обработки изображений прочие форматы с повышенной точностью, вывод на экран, как правило, осуществляется в формате TrueColor. При этом стандартным цветовым пространством является sRGB. Цветовое пространство определяет, какой реальный цвет соответствует каждому конкретному числовому значению цвета. И хотя пространство sRGB покрывает всего около 35% всех видимых человеком цветов (Рис. 6), глубины цвета TrueColor не хватает для отображения всех различных цветов даже внутри sRGB. Наибольшие проблемы возникают с отображением плавных градиентов, так как в некоторых случаях явно видны границы между соседними цветами. Данная проблема усугубляется при использовании более широких цветовых пространств, например AdobeRGB, где доступные в каждом цветовом канале 256 уровней еще сильнее отдаляются друг от друга.

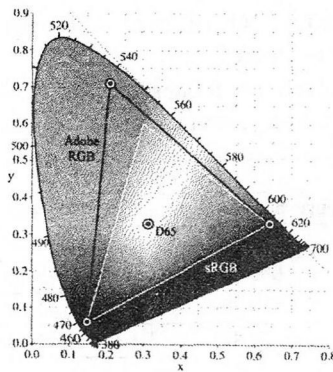


Рис. 6: Сравнение цветовых пространств AdobeRGB и sRGB на хроматической диаграмме CIE xy.

Однако этап деквантования цветов при декодировании в большинстве схем сжатия можно проводить с использованием выходных форматов с любой глубиной цвета.

Для определения максимальной глубины цвета n , которую разумно использовать для конкретной схемы сжатия, предлагается оценивать максимальную ошибку сжатия e для равномерного вертикального или горизонтального

градиента с шагом c_n , минимальным для данной глубины цвета. В таком случае, условие $e < c_n$ является критерием возможности сжатия текстур с глубиной цвета n с помощью данного формата сжатия. Дальнейшее увеличение глубины цвета, приведёт к тому, что пары значений, различающихся на "1" не смогут быть различены в сжатом блоке.

Для оценки максимальной глубины цвета для форматов семейства DXT/BC предлагается использовать формулу $n = (S_c + S_i - 1)$, где S_c — точность представления цвета в сжатом блоке, а S_i — размер в битах отдельного индекса. Исходя из этого, формат BC7, а в частности тип блока BC7 Mode6, потенциально может использоваться для хранения текстур с глубиной цвета до 10 бит.

В работе предлагается новый режим работы декодера, в котором деквантование базовых цветов и последующие операции будут производиться с точностью 10 бит, а также соответствующая модификация параллельного кодера. Формат сжатого блока при этом остаётся неизменным, и текстуры остаются обратно совместимыми с существующими декодерами, которые смогут корректно распаковать блок с использованием глубины цвета 8 бит.

Для практической проверки состоятельности предложенного метода было произведено тестирование качества сжатия различных текстур с глубиной цвета 10 бит с использованием программного кодера. За основу был взят кодек `bc7_gri`, производящий сжатие в формат BC7 на графическом процессоре с использованием технологии OpenCL.

Значения метрики PSNR (Peak Signal to Noise Ratio, пиковое соотношение сигнал/шум) для тестовых текстур приведены в таблице 3.

	PSNR		Разница
	Оригинальный кодек	Предлагаемый кодек	
Текстура № 1	51,99 dB	53,12 dB	1,13 dB
Текстура № 2	51,80 dB	52,78 dB	0,99 dB
Текстура № 3	53,70 dB	55,45 dB	1,74 dB
Текстура № 4	51,20 dB	51,63 dB	0,43 dB

Таблица 3: Качество сжатия текстур с глубиной цвета 10 бит

Результаты тестирования показали, что предлагаемая модификация ощутимо увеличивает качество сжатия текстур, имеющих плавные градиенты с глубиной цвета 10 бит. Увеличение значения метрики PSNR составляет 0.4–1.7 dB.

В пятой главе рассматривается последовательно-параллельная модификация текстурного кодера ASTC для работы на гетерогенных системах, объединяющих ЦП и ГП.

На сегодняшний день имеется только один кодек формата ASTC — ASTC Evaluation Codes. Из-за гибкости формата, алгоритм сжатия является наиболее сложным и трудозатратным среди остальных форматов сжатия. Для повышения производительности в алгоритме используется большое число эвристик и условий раннего выхода, что делает неэффективным перенос всего алгоритма на ГП. С другой стороны, выполнение на ГП только лишь небольших хорошо распараллеливаемых заданий связано с большими накладными расходами на копирование данных и диспетчеризацию заданий.

Для ускорения процесса сжатия, в работе предлагается модификация алгоритмов сжатия для работы на гетерогенных системах класса HSA, где универсальные и графические процессоры имеют доступ к общей виртуальной и физической памяти, а перенос вычислений с одного типа процессоров на другой имеет малые накладные расходы. Предлагается выделить в последовательном алгоритме сжатия отдельного блока секции, внутри которых возможна параллельная когерентная многопоточная обработка, и вычисления внутри таких секций проводить с использованием ядер ГП. В таком случае, ускорение работы алгоритма возможно за счёт двух факторов. Во-первых, параллельные секции способны существенно быстрее исполняться на ГП ядрах. Одновременно снижаются и энергозатраты, так как ГП ядра значительно эффективнее в плане удельного энергопотребления на Gflops. Во-вторых, за счёт параллельности на уровне блоков, процессорные ядра могут исполнять последовательные секции для других блоков, в то время как графические ядра исполняют параллельные секции.

Для реализации параллельных частей алгоритма характерны те же самые ограничения, как и для остальных алгоритмов, предназначенных для работы на ядрах ГП. Однако у программ в гетерогенной среде, по сравнению с теми, что работают только на одном типе процессоров, появляются дополнительные особенности. Хотя архитектура HSA резко снижает расходы на вызов вычислительных фрагментов на ядрах ГП, они всё равно остаются значительными. Поэтому снижение общего количества взаимодействий ЦП↔ГП является важной задачей.

Для решения данной проблемы, предлагается в рамках одного ЦП потока обрабатывать сразу множество блоков (далее “пакет”). Пусть обработка одного блока состоит из трёх секций А, В, С, и секция В допускает параллельную обработку. Тогда суть изменения можно схематично показать на Рис.7. Такой подход позволяет повысить загрузку ядер ГП. Однако объём памяти, необходимый для вычислений, также повышается, так как необходимо хранить промежуточные данные, передаваемые между секциями, для всех блоков пакета.

Размер пакета оказывает значительное влияние на скорость сжатия. Чем больше блоков входит в один пакет, тем меньше общее количество вызовов вы-

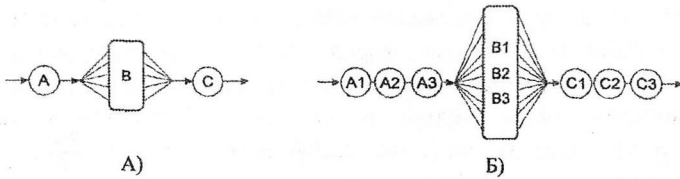


Рис. 7: Схема обработки блоков отдельно (А) и пакетно (Б)

числительных ядер на ГП. Также больший размер заданий позволяет эффективнее использовать вычислительные ресурсы ГП. Эксперименты показали (Рис. 8), что пакеты размером 128–512 блоков являются наиболее предпочтительными, так как обеспечивают достаточную загрузку ядер ГП и имеют умеренные требования к объёму памяти. Начиная с 256 блоков, дальнейшее увеличение размера пакета практически не влияет на скорость сжатия. При размере пакета менее 64 блоков время сжатия резко возрастает, что объясняется недостаточной загрузкой ядер ГП.

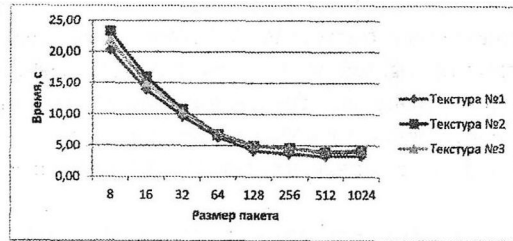


Рис. 8: Скорость сжатия текстур для различных размеров пакета (режим сжатия "medium", CPU AMD A10-7850K)

Для повышения производительности в работе также предлагается использование компиляции во время исполнения (JIT-компиляция). Часть параметров сжатого блока, таких как размер блока в пикселях, а также параметров качества, ограничивающих пределы поиска решений, задаются пользователем и имеют одинаковое значение для всех блоков изображения. Поэтому вместо передачи таких параметров в качестве аргументов функциям, исполняемым на ГП, перед началом процесса сжатия можно перекомпилировать такие функции, подставляя в текст программы уже известные на этот момент параметры, т.е. использовать JIT-компиляцию для таких функций. Это позволяет компилятору провести дополнительные оптимизации бинарного кода. За счёт этого уменьшается общий объём бинарного кода, появляется возможность развёртывания большего количества циклов, сокращается расход регистров, что позволяет раз-

местить больше контекстов SIMD-векторов на одном ядре ГП. Пример результатов компиляции функции `find_best_partitioning()` для ГП AMD Radeon R9 290x (архитектура GCN) приведён в Таблице 4.

Параметр	Статическая компиляция	ЛПТ-компиляция
Размер бинарного кода	24124 байт	7628 байт
Расход векторных регистров	65	36
Расход скалярных регистров	76	50

Таблица 4: Сравнение результатов работы статической и ЛПТ-компиляции

Результаты тестирования (Таблица 5) показали увеличение быстродействия в 3–5 раз без снижения визуального качества результирующей текстуры.

Текстура	Режим сжатия	Время сжатия		Ускорение
		Оригинальный кодек	Предлагаемый кодек	
Текстура №1	Medium	10.2 с	3.2 с	3.19x
Текстура №2		13.2 с	3.9 с	3.41x
Текстура №3		12.6 с	4.0 с	3.19x
Текстура №1	Thorough	43.1 с	9.9 с	4.36x
Текстура №2		41.9 с	9.6 с	4.35x
Текстура №3		36.0 с	8.8 с	4.08x
Текстура №1	Exhaustive	99.5 с	19.8 с	5.04x
Текстура №2		87.7 с	17.9 с	4.90x
Текстура №3		91.4 с	17.6 с	5.19x

Таблица 5: Сравнение быстродействия оригинального и предлагаемого кодеков формата ASTC (CPU — AMD A10-7850K)

Заключение. Главный научный результат диссертационной работы заключается в разработке методов компрессии текстур, позволяющих производить сжатие текстур непосредственно на графическом процессоре, методов сжатия текстур, работающих в гетерогенных системах, а также в создании соответствующих программных средств сжатия текстур, обеспечивающих существенно большее быстродействие, чем аналогичные алгоритмы, исполняемые на ЦП.

К основным результатам диссертационной работы относятся следующие:

1. Систематизированы и детально описаны все современные стандартные форматы сжатия текстур, что представляет ценность для учёных и инженеров, работающих в сфере текстурной компрессии. Соответствующая статья на русском и английском языках опубликована в журнале индексируемом в базе SCOPUS.
2. Разработаны методы снижения накладных расходов на межкадровое сжатие текстур на ГП за счёт ликвидации избыточного этапа копирования, а также модификации тайлового конвейера.
3. Разработана модификация алгоритма и программная реализация быстрого сжатия текстур в формате DXT/BC1, работающая на графическом процессоре и обеспечивающая повышение быстродействия на 2 порядка, по сравнению с многопоточным CPU-вариантом.
4. Разработана модификация схемы сжатия текстур BC7 для хранения текстур с глубиной цвета 10 бит за счёт изменения этапа деквантования, с соответствующей модификацией кодека. Результаты экспериментов со сжатием текстур, имеющих плавные градиенты, показали увеличение показателя PSNR на 0.4–1.7 dB.
5. Разработана модификация алгоритма и программная реализация сжатия текстур в формате ASTC для исполнения в гетерогенных системах в рамках проекта корпорации AMD. Результаты экспериментов показали увеличение производительности по сравнению с оригинальным кодеком в 3–5 раз без снижения визуального качества результирующей текстуры.

Публикации автора по теме диссертации

1. Перминов И.В., Палташев Т.Т. Гетерогенная архитектура для CPU, GPU и DSP / Открытые Системы. СУБД. — 2013. — Платформы. — С.12-15. — 63 с (из перечня ВАК)
2. Перминов И.В. Повышение эффективности обработки динамически сжимаемых текстур // Научно-Технический Вестник Информационных технологий, механики и оптики. — 2013. С. 164-165 (из перечня ВАК)
3. Перминов И.В., Палташев Т.Т. Использование контейнера BC7 для хранения текстур с глубиной цвета 10 бит // Научно-технический вестник информационных технологий, механики и оптики. — Санкт-Петербург, 2014. — Вып. №1 (89). — Компьютерные системы и информационные технологии. — С. 107-112 (из перечня ВАК)
4. Perminov I.V., Paltashev T.T. Texture compression techniques // Scientific Visualization, National Research Nuclear University "MEPhI". — 2014. — Т. 6, вып. 2014-1. — С. 96-136 (из перечня SCOPUS)
5. Перминов И.В., Герасимов А.А. Взаимодействие GPU и CPU на кристалле с использованием разделяемой памяти // Сборник трудов молодых ученых и сотрудников кафедры ВТ. — Санкт-Петербург, 2012. — Вып. Выпуск 3. — Компьютерные системы и сети. — С. 39-43
6. Перминов И.В. Динамическая компрессия визуальных данных в графических процессорах // Параллельные вычисления и задачи управления РАСО'2012 Труды шестой международной конференции. — Москва, 2012. — Архитектура параллельных и распределенных систем. — С. 31-41
7. Перминов И.В., Герасимов А.А., Баенова Г.М. Реализация компрессии текстур в темпе генерации кадров с использованием OpenCL // Сборник тезисов докладов I всероссийского конгресса молодых ученых. — 2012. — Вып. Выпуск 1. — С. 55-56
8. Перминов И.В., Палташев Т.Т. Гетерогенные архитектуры: экосистема для CPU/GPU/DSP // Тезисы докладов Четвертого Московского Суперкомпьютерного Форума. — 2013. — С. 30-31
9. Модификация алгоритма текстурной компрессии LSDxt для работы на графическом процессоре // Сборник тезисов докладов конгресса молодых ученых. — Санкт-Петербург, 2013. — Вып. Выпуск 1. — С. 47. — 193 с

10. Особенности использования регистровой памяти GPU в OpenCL на примере текстурного компрессора // 23-я Международная Конференция по Компьютерной Графике и Зрению ГрафиКон'2013 Труды Конференции. — Владивосток, 2013. — С. 267-270

Тиражирование и брошюровка выполнены в учреждении
«Университетские телекоммуникации»
197101, Санкт-Петербург, Саблинская ул., 14
Тел. (812) 233 46 69.
Объем 1,0 у.п.л. Тираж 100 экз.